



FA–QABC–MRTA: a solution for solving the multi-robot task allocation problem

Farouq Zitouni¹ · Ramdane Maamri² · Saad Harous³

Received: 21 July 2018 / Accepted: 26 August 2019 / Published online: 14 September 2019
© Springer-Verlag GmbH Germany, part of Springer Nature 2019

Abstract

The problem of task allocation in a multi-robot system is the situation where we have a set of tasks and a number of robots; then each task is assigned to the appropriate robots with the aim of optimizing some criteria subject to constraints, e.g., allocate the maximum number of tasks. We propose an effective solution to address this problem. It implements a two-stage methodology: first, a global allocation based of the well-known firefly algorithm, and then, a local allocation combining advantages of quantum genetic algorithms and artificial bee colony optimization. We compared our proposed solution to one solution from the state of the art. The simulation results show that our scheme significantly performs better than this solution. Our solution allocated 100% of the tasks (in every configuration tried in the experiments) and enhanced the allocation time by 75%.

Keywords Multi-robot system · Task allocation · Firefly algorithm · Artificial bee colony optimization · Quantum genetic algorithms

1 Introduction

Nowadays, multi-robot systems (MRS) are almost in every aspect of our daily lives, and intensive research is being done to achieve their opportunities. The main motivation behind such attention is because they make it easy to solve many complex problems, such as industrial applications, surveillance, and rescue missions in environments hit by natural disasters [11].

Also, it must be mentioned that the design of a MRS must consider interactions between its components—i.e., robots; otherwise, it risks to produce a system with limited and non-

deterministic performance [10]. Thereby, one of the major coordination problems that researchers must solve in this field is the task allocation—named multi-robot task allocation (MRTA) [9]. In the next section, we define the MRTA problem.

1.1 Definition of the MRTA problem

The MRTA problem can be defined as follows: We assume that we have some tasks to be performed and robots that are capable of achieving these tasks. Consider tasks to be allocated, which robots are adequate to execute them. These assignments must optimize the criterion taken into account, e.g., minimize completion time or makespan. So, the goal is to coordinate robots' behaviors and find an optimal way to allocate all tasks [15]. The basic mathematical formulation of this problem is described in Table 1 [27].

Also, other constraints may be considered, e.g., spatial, energetic, and temporal. In linear programming, this formulation is called assignment problem—can be easily solved using a suitable algorithm, e.g., Hungarian method. Generally, exact methods do not capture constraints imposed by real-life problems, and hence, there is a need to rethink them for the treatment of complex cases.

✉ Farouq Zitouni
farouq.zitouni@univ-constantine2.dz

Ramdane Maamri
ramdane.maamri@univ-constantine2.dz

Saad Harous
harous@uaeu.ac.ae

¹ Department of Computer Science, Kasdi Merbah University, Ouargla, Algeria

² Department of Computer Science, Constantine 2 - Abdelhamid Mehri University, Constantine, Algeria

³ Department of Computer Science, UAE University, Abu Dhabi, United Arab Emirates

Table 1 The basic mathematical formulation of MRTA problem

Optimize	$\sum_{i \in N} \sum_{j \in M} x_{ij} u_{ij}$	
Subject to		
1)	$\sum_{i \in N} x_{ij} = 1,$	$\forall j \in M$
2)	$\sum_{j \in M} x_{ij} = 1,$	$\forall i \in N$
3)	$x_{ij} \in \{0, 1\},$	$\forall i \in N, \forall j \in M$

Here N : robots' set, M : tasks' set, u_{ij} : criterion to be optimized

A task is a sub-goal of the system it belongs to. It is executed in a dependent manner—e.g., after another task, or in an independent manner—i.e., no constraint is imposed. It is discreet—e.g., move an object from the point A to the point B , continuous—e.g., monitor the entry of a bank, or hybrid—e.g., move an object from the point A to the point B while monitoring its environment to avoid collisions. To allocate tasks, every robot must calculate a numerical value for each one. Intuitively, this measurement is called utility—i.e., it expresses cost, gain, distance, time, or their combination.

1.2 Definition of the utility

In optimization, solving the MRTA problem comes to find assignments between tasks and robots, while optimizing a considered numerical measurement. Commonly, this value is named utility or fitness [6]. To approach this concept, we give tasks that must be allocated to some robots. We assume that every robot is capable to compute an utility value for each task—noted u_{ij} . Thus, the utility of a robot r for a task t is computed by Eq. (1).

$$u_{rt} = \begin{cases} q_{rt} - c_{rt}, & \text{if } r \text{ is capable of executing } t \\ \infty, & \text{otherwise} \end{cases} \quad (1)$$

where q_{rt} : quality of performing t by r , c_{rt} : cost of performing t by r .

Also, it is important to note that the utility of a robot for a task can depend on other robots' utilities of the same environment [19]. Thereby, the previous equation can be generalized to manipulate robots' and tasks' subsets. We assume that R and T are two robots' and tasks' subsets— $|R| \geq 1$ and $|T| \geq 1$. Thus, Eq. (1) can be reformulated as follows.

$$U_{RT} = \begin{cases} Q_{RT} - C_{RT}, & \text{if robots in } R \text{ are capable of executing tasks in } T \\ \infty, & \text{otherwise} \end{cases} \quad (2)$$

where Q_{RT} : quality of performing tasks in T by robots in R . C_{RT} : cost of performing tasks in T by robots in R .

In practice, we find several variants of MRTA problems and chosen algorithms to resolve them depend on the faced configuration. In the next section, we give three taxonomies used for the classification of MRTA problems.

1.3 Taxonomies for MRTA problems

In the literature, we find three taxonomies to categorize MRTA problems. Their purpose is to propose a classification of task allocation situations into classes, so that real-life problems can be projected onto one of them—often a mathematical formulation is given. For simplicity, we name these taxonomies: taxonomy A, B, and C, respectively.

1.3.1 Taxonomy A

The authors of [6] categorize MRTA problems by proposing a classification which is articulated around three axes. The first one distinguishes robots according to their abilities to be assigned either to a single task or several tasks at a time. The second one distinguishes tasks according to their need to be allocated either to a single robot or to several robots at the same time. The third one takes into account how tasks are assigned to robots considering either only current or both current and future information.

The authors [6] defined two points at each axis as follows. Axis 1: single-task robots (ST), multi-task robots (MT), axis 2: single-robot tasks (SR), multi-robot tasks (MR), and axis 3: instantaneous assignment (IA), time-extended assignment (TA). Thus, eight problem classes are distinguished, i.e., ST-SR-IA, ST-SR-TA, ST-MR-IA, ST-MR-TA, MT-SR-IA, MT-SR-TA, MT-MR-IA, and MT-MR-TA. Despite its good coverage of most MRTA problems, however, this taxonomy does not capture problems with interrelated utilities, and temporal constraints on tasks and robots [6]. Hence, there is a need to propose a taxonomy that addresses these limitations.

1.3.2 Taxonomy B

The authors of the work [19] modified taxonomy A by adding a higher level which expresses interdependence degrees between tasks and robots. This taxonomy is a hierarchy with two levels. The second one includes the classes proposed in Taxonomy A. The first one encompasses four interdependence degrees between tasks and robots as follows. One, “No Dependencies (ND)”: The utility of a robot for a task does not depend on anything—it covers ST-SR-IA and ST-SR-TA problem classes. Two, “In-Schedule Dependencies (ID)”: The utility of a robot for a task depends only on its schedule—it covers ST-SR-TA, MT-SR-IA, and MT-SR-TA problem classes. Three, “Cross-Schedule Dependencies (XD)”: The utility of a robot for a task does not only depend on its schedule but also on other robots' schedules, and schedules are static—it covers the eight problem classes. Four, “Complex Dependencies (CD)”: The utility of a robot for a task does not only depend on its schedule and on other robots' schedules, but in this case schedules are dynamic—it covers the eight problem classes.

1.3.3 Taxonomy C

The authors of [26] have modified taxonomy A by splitting the “time-extended assignment (TA)” point into two sub-points. This action allows the introduction of temporal constraints on tasks as follows: first, the sub-point “TA:TW” which allows to consider temporal constraints in the form of time windows; second, the sub-point “TA:SP” which makes it possible to consider synchronization and precedence constraints. In the next section, we overview previous work that tackled the MRTA problem.

1.4 Related work

We discuss previously proposed approaches to address the MRTA problem. These solutions are listed taking into consideration the first level of taxonomy B, i.e., they are divided into four groups. It should be noted that this list is not exhaustive; however, it covers those frequently cited in the literature [15].

In the first group, we find several approaches in the literature [6]. Particularly, they dealt with ND[ST–SR–IA] problems. We mention the work in Vail and Veloso [35] that uses potential fields, and works in Gerkey and Mataric [7], Simmons et al. [32] that use auction-based approaches. In the second group, we find several works. Generally, they focused on ID[ST–SR–TA] problems. The work in Barnhart et al. [1] uses TSP and mTSP to treat this problem. The solution provided in Melvin et al. [25] is a good solution for the routing problem using mixed linear programming. Due to their distributed nature, auction-based approaches are also used to address this problem [2,5,16,20].

There are several research papers among the third group. For example, M+ system [3] which addressed the instantaneous task allocation by using a market system with precedence constraints. The work [24] proposes a solution that manipulates constraints on tasks and a market-economy method. The work [4] addresses the routing problem by proposing three methodologies—robots’ teams must perform scientific missions. A solution that uses quantum genetic algorithms and reinforcement learning is proposed in Zitouni and Maamri [39]. The case of simple task scheduling is considered in Lemaire et al. [22] which uses the heuristic: “task x must be accomplished n second(s) before the starting of task y .” Another way to solve this problem is to bring it to a coalition formation problem [29,30]. The approach proposed in Vig and Adams [36] enhances the works in Shehory and Kraus [29,30] by minimizing communication rates and imposing constraints on agents’ capabilities. The authors of Guerrero and Oliver [8,23] use auctions and coalitions to address this problem. The Framework [31] proposes a solution that imposes the constraint of shared resources, i.e., communication mediums and processors. Finally, the solutions proposed in Koes et al. [17], Korsah [18] addressed

the problem where robots must consider current and future information for task allocations.

In the fourth group, there are fewer works because of the increased difficulty of the problems and the lack of mathematical formulations [19]. The work [12] addresses this problem while considering both current and future information for task allocations—the approach has been applied to manage an environment hit by a natural disaster. The approach presented in Parker and Fang [28] uses coalition formation—this solution was originally named ASyMTRe in its centralized version and latter ASyMTRe-D in its distributed version. Finally, we mention the work [41] that modified [5] and dealt with both current and future assignments.

1.5 Paper organization

The remainder of the paper is organized as follows. In Sect. 2, we present the proposed solution to treat this problem and explain its steps. In Sect. 3, we simulate our solution, compare it to another approach [40], and discuss the obtained results. In Sect. 4, we give a conclusion and some perspectives.

2 Proposed solution

An auction-based solution, named FA–QABC–MRTA: fire-fly algorithm–quantum artificial bee colony–multi-robot task allocation, is presented to solve the MRTA problem. Auction-based methods are the most used to solve this problem due to their advantages, e.g., the simplicity of cost function satisfaction [34]. These approaches are based on auctions, which consist of an auctioneer, bidders, and goods. If a MRTA problem is taken into account, a particular robot is an auctioneer, rest of robots are the bidders, and tasks are the goods. For communication, we utilize the contract net protocol [33].

2.1 Roles of system actors and assumptions

The auctioneer communicates tasks to be allocated to the bidders and receives their offers. After that, it decides the best assignments between bidders and tasks and finally notifies the concerned ones. Although the use of an auctioneer can be seen as a bottleneck—e.g., if it breaks down then the system will be broken too, but it greatly reduces the number of exchanged messages and keeps a global view of the current system state. Also, this solution has the advantage of calculations’ sharing on bidders (scalability), which would make it possible to maintain the progression of assignments even if one of them breaks down (robustness) [34].

The bidders calculate utilities for tasks based on information they have. So, the overall cost function is divided into sub-functions—estimated in a decentralized and independent manner by bidders. In our solution, the execution of

a task simply means the presence of resources it requires—i.e., a task is said to be executed if and only if resources it requires are all available. The resources are made available by bidders, and are usually sensors and actuators.

We assume the class of problems ST-MR-IA, i.e., each bidder is allocated at most to one task at a time, some tasks are allocated to several bidders at a time, and tasks are assigned to bidders considering only current information. We give two non-empty sets B and T —bidders and tasks, respectively. We suppose that all tasks are known beforehand or discovered gradually. Each bidder $b \in B$ is defined by a sextuplet $\langle R_b, C_b, v_b, e_b, a_b, p_b \rangle$. The parameter R_b is a binary vector representing its resources, and C_b is a vector representing resource costs. The other parameters represent its speed, energy level, aging factor, and spatial position, respectively. Each task $t \in T$ is defined by a triplet $\langle R_t, d_t, p_t \rangle$. The parameter R_t is a binary vector representing resources it requires. The other parameters represent its duration and spatial position, respectively. Next, we present and explain the different algorithms proposed to address the MRTA problem.

2.2 Actors' behaviors

The following algorithms show behaviors of the auctioneer and bidders. Algorithms 1 and 2 have two sub-behaviors—i.e., “Periodic Behavior” and “Cyclic Behavior.” The first one includes the operations that are run periodically—i.e., every n seconds. The second one includes the operations that are executed after a message is received. These behaviors are explained in the following sections.

The task allocation process begins when the auctioneer announces to bidders the availability of tasks to be allocated—“Periodic Behavior” of Algorithm 1. To do this, it broadcasts a message to all bidders—which contains the resources required by tasks and their position.

If a message is received—“Cyclic Behavior” of Algorithm 2, each bidder chooses a task to execute, calculates its cost, and replies to the auctioneer—“Periodic Behavior” of Algorithm 2. To choose a task, a bidder executes the firefly algorithm [37,38] on received tasks, and gradually converges to one of them. To converge to a task, we use Eqs. (3) and (4). Once a task has been selected, the bidder estimates its cost for each resource it offers using Eq. (5). Finally, the bidder sends its result to the auctioneer. The symbol $\perp$ means that the bidder sends a special notification to the auctioneer informing it that it did not select any task.

$$p_b = p_b + e^{-\gamma \times (d_{bt})^2} \times (p_t - p_b) + \alpha(\text{rand} - 0.5) \quad (3)$$

$$d_{bt} = \delta \|p_t - p_b\| + (1 - \delta)H(R_t, R_b) \quad (4)$$

$$u_{r \in \Delta_{bt}}^{bt} = \left[\delta c_r^b + (1 - \delta) \left(\frac{v_b}{\|p_t - p_b\|} \right)^{\frac{e_b}{a_b}} \right] \times |\Delta_{bt}| \quad (5)$$

Algorithm 1: the auctioneer behavior.

```

1 While The Auctioneer Is Alive
2   responses  $\leftarrow \emptyset$ ; {It contains bidders' responses.}
3   {This behavior is executed periodically.}
4   Periodic Behavior
5     {If the condition is met, it means that some tasks are
6     available for allocation.}
7     if ( $|T| \neq 0$ ) then
8       foreach  $b \in B$  do
9         {A message contains the receiver, subject, and
10        content, respectively.}
11        sendMessageTo( $b$ , "CHOOSE-A-TASK",  $T$ );
12      end
13    end
14  Cyclic Behavior
15    {This behavior is executed after a message is received.}
16    responses  $\leftarrow$  responses  $\cup$  {receiveMessageFrom( $b$ )};
17    {If the condition is met, it means that all bidders'
18    responses are received.}
19    if (responses.isAll() = true) then
20      foreach  $t \in T$  do
21        if ( $t.canBeAllocated()$  = true) then
22          { $B'$  is a set of robots ( $B' \subseteq B$ ).}
23          {An allocation is computed using Algorithm
24          3.}
25           $B' \leftarrow$  computeAnAllocationFor( $t$ );
26           $T \leftarrow T / \{t\}$ ;
27          foreach  $b \in B'$  do
28            sendMessageTo( $b$ , "NOTIFY",  $t$ );
29          end
30        end
31      end
32    responses  $\leftarrow \emptyset$ ;
33  end

```

where p_b : position of robot b . p_t : position of task t . $e^{-\gamma \times (d_{bt})^2} \times (p_t - p_b)$: attraction degree of b by t . γ : attraction variation—its value is very important for the algorithm convergence speed and the behavior of b ($\gamma \in [0.01, 100]$). α : environment noise—its value affects the visibility of tasks by bidders ($\alpha \in [0, 1]$). rand: a random value (rand $\in [0, 1]$). d_{bt} : weighted distance between b and t , it is given by Eq. (4). $\|p_t - p_b\|$: Euclidean distance between b and t . $H(R_t, R_b)$: Hamming distance between the vectors R_t and R_b . δ : parameter serves to balance the two distances ($\delta \in [0, 1]$). Δ_{bt} : common resources between b and t —i.e., the resources offered by b to t .

When all bidders' responses are received, the auctioneer calculates an allocation for each selected task—“Cyclic Behavior” of Algorithm 1. To do this, it considers for each task all the resources offered by the bidders, then it combines the artificial bee colony and quantum genetic algorithms to calculate an allocation for it. At the end of this stage, bidders

Algorithm 2: the bidder behavior.

```

1 While The Bidder Is Alive
2    $T \leftarrow \emptyset$ ; {It contains tasks available for allocation.}
3   {This behavior is executed periodically.}
4   Periodic Behavior
5     if ( $|T| \neq 0$ ) then
6       if ( $state = "AVAILABLE"$ ) then
7         { $n$  is the number of iterations.}
8         for  $i \leftarrow 1$  to  $n$  do
9           foreach  $t \in T$  do
10            Converge the bidder to the task  $t$  using
            Eq. (3) and (4).
11          end
12        end
13         $t \leftarrow$  choose the closest task to the bidder;
14         $c \leftarrow$  estimate costs of offered resources using
        Eq. (5);
15        {A message contains the subject, chosen task, and
        costs, respectively.}
        sendMessageToTheAuctioneer("CHOSEN-
        TASK",  $t, c$ );
16      else
17        {An empty message is sent.}
18        sendMessageToTheAuctioneer("CHOSEN-
        TASK",  $\perp, \perp$ );
19      end
20       $T \leftarrow \emptyset$ ;
21    end
22  end
23  {This behavior is executed after a message is received.}
24  Cyclic Behavior
25    message  $\leftarrow$  receiveMessageFromTheAuctioneer();
26    if ( $message.isEmpty() \neq true$ ) then
27      if ( $message.getSubject() = "CHOOSE-A-TASK"$ )
28      then
29         $T \leftarrow message.getContent()$ ;
30      end
31    end
32  end

```

concerned by the found allocations are notified. Algorithm 3 presents the steps of this combination.

2.3 ABC and QGA for task allocation

We explain the instructions of Algorithm 3—we use an illustrative example and clarify algorithm equations. It hybridizes advantages of two methods, well known for their efficiency, frequently used to solve optimization problems, artificial bee colony [13] and quantum genetic [21] algorithms.

Figure 1 depicts a simple scenario of a MRTA problem. In this example, we want to allocate Task 1—it requires four sensors, i.e., touch, sound, light, and ultrasonic. These sensors are offered by six bidders, i.e., Robot 1, Robot 2, Robot 3, Robot 4, Robot 5, and Robot 6. To each bidder sensor is assigned a real valuation representing its cost—calculated by

Algorithm 3: ABC and QGA for task allocation.

```

1 Randomly initialize a quantum population  $F_{ij}^p$  of  $n$  food sources
  using Eq. (6);
2  $i \leftarrow 0$ ;
3 Make  $M_{ij}^p$  using Eq. (7) – measure of every food source
   $F_{ij}^p \rightarrow M_{ij}^p$ ;
4 Evaluate every  $M_{ij}^p$  using constraints  $c1, c2, c3$ , and  $c4$  – fitness
  function;
5 Memorize the best food source;
6 while stopping criteria is not met do
7   Employed Bee Stage
8     Perform an update process for each food source using
      Eq. (9);
9   end
10  Onlooker Bee Stage
11    Randomly select food sources using Eq. (10);
12    Perform an update process for each food source using
      Eq. (9);
13  end
14  Rotation;
15  Mutation;
16   $i \leftarrow i + 1$ ;
17  Make  $M_{ij}^p$  using Eq. (7) – measure of every food source
     $F_{ij}^p \rightarrow M_{ij}^p$ ;
18  Evaluate every  $M_{ij}^p$  using Eq. (8) – fitness function;
19  Memorize the best food source;
20  Scout Bee Stage
21    Select one of the most inactive food sources;
22    Replace it by a new randomly generated one using Eq. 6;
23  end
24 end

```

Eq. (5). Therefore, the objective is to find a subset, i.e., an allocation, satisfying the following three constraints.

1. The cost of this allocation must be minimal among all allocations,
2. The cardinality of this allocation is equal to the number of sensors required by the task taken into account—in the previous example, the cardinality is equal to 4, and
3. Resource repetitions in an allocation are not allowed, i.e., only one instance of each resource is required.

Generally, this kind of problems can be solved easily and optimally in linear programming—we just use an exact method. However, if the number of bidders and resources is large, then exact methods become very quickly ineffective—with the previous example, we have to consider $2^{4+3+5+6}$ different subsets and choose the allocation that simultaneously satisfies the previous constraints and its cost is optimal.

As a result, we propose to combine the powers of ABC and QGA to address this optimization problem. The MRTA problem being addressed here can be modeled using the formulation illustrated in Table 2. The term $u_{r \in \Delta_{bt}}^{bt}$ represents costs of resources offered by bidders. R_i is the set of resources

Fig. 1 A scenario of a MRTA example

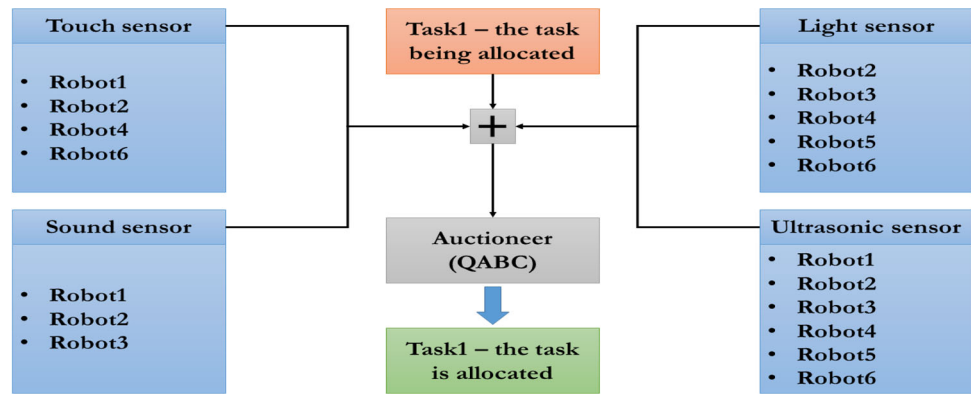


Table 2 The mathematical formulation of our MRTA problem

Min	$\sum_{r \in R_t} u_r^{bt}$			
Subject to				
1)	$\forall t \in T$	$\forall b \in B$		$u_r^{bt} \in [l_r, u_r]$
2)	$\forall t \in T$	$\forall b, b' \in B$	$\forall r, r' \in R_t$	$(x_r^{bt} - x_{r'}^{b't}) = 0 \rightarrow r \neq r'$
2)	$\forall t \in T$	$\forall b \in B$	$\forall r, r' \in R_t$	$(x_r^{bt} - x_{r'}^{bt}) = 0 \rightarrow r \neq r'$
2)	$\forall t \in T$	$\forall b \in B$		$\sum_{r \in R_t} x_r^{bt} = R_t $
4)	$\forall t \in T$	$\forall b \in B$	$\forall r \in R_t$	$x_r^{bt} \in \{0, 1\}$

needed by the task t . Constraint 1 gives the range of allowed values for costs. Constraints 2 and 3 ensure that two bidders never offer the same resource at the same time. Constraint 4 ensures that the cardinality of an allocation is equal to the number of resources required by the task. Constraint 5 is an indicator: if bidder b offers a resource r to a task t then its value is 1, otherwise its value is 0.

2.3.1 Initialization of individuals

A quantum population of food sources is initialized—i.e., individuals, candidate solutions, or allocations. Each individual contains several quantum chromosomes—i.e., the number of resources required by the task to be allocated. The number of genes in each chromosome is equal to the number of bidders that offered the corresponding resource—chromosomes' lengths are not necessarily identical. Figure 2 shows the structure of an individual, i.e., a candidate solution, of the previous example. We consider N the population size.

This individual contains four chromosomes—the task requires four resources, and the chromosomes' lengths are not equal. In general, a quantum chromosome is a $2 \times m$ matrix, where m is the number of genes in each chromosome. For example, the chromosome 3 in Fig. 2 is a matrix 2×5 —we have five robots that offer the corresponding resource, i.e., the light sensor. Equation (6) is used for individuals' initialization [21].

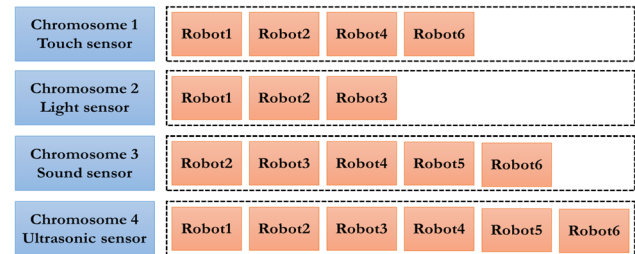


Fig. 2 An example of an individual

$$\begin{bmatrix} \alpha_1 & \dots & \alpha_m \\ \beta_1 & \dots & \beta_m \end{bmatrix} = \begin{bmatrix} \left(\begin{matrix} \cos(\theta_1) & -\sin(\theta_1) \\ \sin(\theta_1) & \cos(\theta_1) \end{matrix} \right) \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ \dots \\ \left(\begin{matrix} \cos(\theta_m) & -\sin(\theta_m) \\ \sin(\theta_m) & \cos(\theta_m) \end{matrix} \right) \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \end{bmatrix} \quad (6)$$

2.3.2 Measurement and evaluation of individuals

To choose the best individual of a population—i.e., the best allocation, first for each individual we project its chromosomes on a binary space and then calculate its fitness value. To project chromosomes of an individual, we use Eq. (7) [21]—this operation is called measurement or observation.

$$\begin{cases} y_i = 0, & m(\alpha) \leq |\alpha_j|^2 \\ y_i = 1, & m(\alpha) > |\alpha_j|^2 \end{cases} \quad (7)$$

where $m(\alpha)$: a random number in the interval $[0, 1]$.

So, each quantum gene $\begin{pmatrix} \alpha_j \\ \beta_j \end{pmatrix}$ becomes a binary gene $y_j \in \{0, 1\}$. Then, the fitness value of each allocation is calculated respecting the following constraints.

- c1: If a binary chromosome q of an allocation p contains several 1—that is: many robots offer the same resource, then its fitness value is $fit_{p,q} = +\infty$.
- c2: If a binary chromosome q of an allocation p does not contain any 1—that is: no robot offers the corresponding resource, then its fitness value is $fit_{p,q} = +\infty$.
- c3: If a binary chromosome q of an allocation p contains exactly 1—that is: one robot offers needed resource, then its fitness value is calculated using Eq. (8) [13]. The parameter u_q^{bt} is the cost of resource q offered by bidder b to task t .

$$fit_{p,q} = \begin{cases} \frac{1}{u_q^{bt}}, & u_q^{bt} \geq 0 \\ 1 + abs(u_q^{bt}), & u_q^{bt} < 0 \end{cases} \quad (8)$$

- c4: Finally, the fitness value of an allocation is equal to the sum of its chromosomes' fitness values. The position of the best allocation is saved.

2.3.3 Hybridization of ABC and QGA operations

We explain the instructions of Algorithm 3 “while” loop. These instructions combine ABC and QGA operations to address the MRTA problem and find the best allocation for a given set of tasks. Steps 1 to 4 are performed during each iteration.

1. In this step (from line 7 to 9), each employed bee calculates a new allocation V_i in the neighboring of its current allocation X_i and evaluates its quality. To do this, Eq. (9) is used. The term X_k is a random allocation. The parameter j is a random index ($j \in [1, N]$). The parameter Φ_{ij} is a random number in the range $[-1, 1]$. If new allocation quality is better than the current one, the employed bee will move toward the new one—using a greedy selection. Otherwise, it stays in its current position.

$$V_{ij} = X_{ij} + \Phi_{ij}(X_{ij} - X_{kj}) \quad (9)$$

2. Once step 1 is finished, the employed bees inform the onlooker bees about allocation qualities. Now (from line 10 to 13), each onlooker bee chooses an allocation using a roulette selection. The probability that an allocation will be selected by onlooker bee is calculated using Eq. (10). The parameter fit_i is the fitness value of the solution. If an allocation is chosen by an onlooker bee, then it will be changed using Eq. (9) as in the previous step.

$$P_i = \frac{fit_i}{\sum_{j=1}^N fit_j} \quad (10)$$

3. In this step (from line 14 to 15), we apply rotation and mutation. These quantum operators make it possible to approach the states of various allocations toward the state of the optimum one, and to disrupt a gene j by swapping positions of its α_j and β_j , respectively. The rotation [21] adjusts, i.e., increases or decreases, gene values of allocations according to the allocation with the lowest fitness value. The mutation [21] makes it possible to explore and exploit new solutions in the search space.
4. The last step (from line 20 to 23) takes place if an allocation is abandoned for a long period of time. Algorithmically speaking, this means that the fitness value of an allocation has not been improved for several generations. If this happens, scout bees generate a new solution using Eq. (6) and replace the abandoned one.

2.4 Quality of found solutions

We discuss the quality of generated solutions. It is important to note that our algorithms use two optimization types, i.e., global and local. First, the global optimization is done by bidders using the firefly algorithm in order to select a task to perform—considering both Euclidean and Hamming distances [Eqs. (3) and (4)]. So, if the parameters of this algorithm are well adjusted, then we can get very good results.

Second, the local optimization is done by the auctioneer combining artificial bee colony optimization—a numerical optimization method simulating the food-seeking behavior of honey bees—and quantum genetic algorithms—a heuristic optimization method simulating human evolutionary mechanisms and principles of quantum physics—in purpose to compute, for each task, an allocation.

Numerical optimization solution based on artificial bee colony is used in successfully solving many real-world applications. When this method is used, the search space is explored in an efficient manner. However, in some cases its exploitation of the search space remains low and its convergence speed is also a problem. A combination of this method and quantum genetic algorithms has been used to improve these limitations. QGA have the superposition principle, which greatly enriches diversity of solutions to be exploited and avoids premature convergence. In conclusion, the found allocations using this hybridization are quite promising, and this is shown in experimental section.

2.5 Handling the nonlinearity of MRTA problems

In multi-robot task allocation problems, we give two distinct sets N and M . The set N represents robots and M denotes

tasks being allocated. First, M is split into a set of subsets $\mathbb{M}$ that jointly verifies the following properties:

- $\mathbb{M}$ is pairwise disjoint: $\forall M_1, M_2 \in \mathbb{M} : M_1 \cap M_2 = \emptyset$ when $M_1 \neq M_2$.
- The union of $\mathbb{M}$ forms the whole set M : $\bigcup \mathbb{M} = M$.
- None of the elements of $\mathbb{M}$ is empty: $\forall T \in \mathbb{M} : T \neq \emptyset$.

The first property indicates that tasks are split into disjoint groups: i.e., no task is assigned to two different groups at the same time. Generally, tasks of the same group share common features (e.g., they are close). In the worst case, each group contains one task. The second property signifies that all tasks must be allocated to robots. The third property denotes that dummy tasks are not considered. Similarly, the set N is also split into a set of subsets $\mathbb{N}$ that jointly verifies the following properties:

- $\mathbb{N}$ is not necessarily pairwise disjoint: $\exists N_1, N_2 \in \mathbb{N} : N_1 \cap N_2 \neq \emptyset$ when $N_1 \neq N_2$.
- The union of $\mathbb{N}$ does not always form the whole set N : $\bigcup \mathbb{N} \neq N$.
- None of the elements of $\mathbb{N}$ is empty: $\forall T \in \mathbb{N} : T \neq \emptyset$.

The first property indicates that robots are split into overlapping groups: i.e., some robots are assigned to two different groups at the same time. In the worst case, each group contains one robot. The second property signifies that not necessary all robots are taken into account to accomplish given tasks. The third property denotes that dummy robots are not considered.

To simplify things, the set of subsets $\mathbb{M}$ is called disjoint partition, and the set of subsets $\mathbb{N}$ is called overlapping partition. It is worth pointing out that the order of elements in $\mathbb{M}$ is very important in MRTA problems, in order to consider all possible allocations. For instance, sets $\mathbb{M}_1 = \{\{m_1\}, \{m_2, m_3\}, \{m_4\}\}$ and $\mathbb{M}_2 = \{\{m_2, m_3\}, \{m_1\}, \{m_4\}\}$ are not equal. Now, we can easily compute the number of all possible disjoint partitions by combining the Bell number [14] and factorial. The following example shows how all disjoint partitions are computed for a given set $M = \{a, b, c\}$. Possible disjoint partitions (without considering order of elements) are:

1. $P_1 = \{\{a\}, \{b\}, \{c\}\};$
2. $P_2 = \{\{a\}, \{b, c\}\};$
3. $P_3 = \{\{b\}, \{a, c\}\};$
4. $P_4 = \{\{c\}, \{a, b\}\};$
5. $P_5 = \{\{a, b, c\}\}.$

We notice that the number of all possible disjoint partitions of any set is easily computed using Eq. (11) (the Bell number). The set $M = \{a, b, c\}$ generates five possible disjoint

partitions. Now, each possible disjoint partition is considered taking into account the order of its elements. For example, the partition P_1 will be considered six times ($3!$, where 3 is the cardinality of P_1). In summary, the set $M = \{a, b, c\}$ has $|P_1|! + |P_2|! + |P_3|! + |P_4|! + |P_5|! = 3! + 2! + 2! + 2! + 1! = 13$ different disjoint partitions. So, the possible disjoint partitions (considering order of elements) are:

1. $P_{1,1} = \{\{a\}, \{b\}, \{c\}\};$
2. $P_{1,2} = \{\{a\}, \{c\}, \{b\}\};$
3. $P_{1,3} = \{\{b\}, \{a\}, \{c\}\};$
4. $P_{1,4} = \{\{b\}, \{c\}, \{a\}\};$
5. $P_{1,5} = \{\{c\}, \{b\}, \{a\}\};$
6. $P_{1,6} = \{\{c\}, \{a\}, \{b\}\};$
7. $P_{2,1} = \{\{a\}, \{b, c\}\};$
8. $P_{2,2} = \{\{b, c\}, \{a\}\};$
9. $P_{3,1} = \{\{b\}, \{a, c\}\};$
10. $P_{3,2} = \{\{a, c\}, \{b\}\};$
11. $P_{4,1} = \{\{c\}, \{a, b\}\};$
12. $P_{4,2} = \{\{a, b\}, \{c\}\};$
13. $P_{5,1} = \{\{a, b, c\}\}.$

$$B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k \quad (11)$$

If we look closely to the previous example, we can easily observe the huge number of possible disjoint partitions, even with a small number of tasks. Note that the number of disjoint partitions is smaller than the number of overlapping partitions. Now, let's $DP(\mathbb{M})$ denotes the set of all possible disjoint partitions, and $OP(\mathbb{N})$ denotes the set of all possible overlapping partitions. To solve the MRTA problem, first we compute the Cartesian product: $DP(\mathbb{M}) \times OP(\mathbb{N})$. In fact, each couple $(\mathbb{M}_i \in DP(\mathbb{M}), \mathbb{N}_j \in OP(\mathbb{N}))$ is called allocation. The allocation that optimizes the considered utility function is the optimal solution. This problem is known to be strongly $\mathcal{NP}$ -hard [6], and it is obvious clear that its algorithm complexity is $\mathcal{O}(n!)$.

Using exact methods to solve MRTA problems is not advocated, particularly when the problem size is large enough: i.e., if the numbers of robots and tasks exceed a given threshold. That is to say, they become ineffective (the larger the size of the problem, the greater its solving time). In order to handle the nonlinearity of MRTA problems and overcome combinatorial explosion, we opt for using metaheuristics. In combinatorial optimization, by searching over a large set of feasible solutions, metaheuristics can often find good solutions with less computational effort than exact methods; however, when a metaheuristic method is used, we may not reach a global optimal solution at the end. Generally, they are useful approaches for optimization problems.

In our solution, we divided the solution process into two stages. In the first one, we used the firefly algorithm. In the

Table 3 Example of bidders' data

ID robot	Location	Resources	Costs	Velocity	Battery level	Aging factor
r_1	(1, 1)	[1, 1, 1, 0]	[8, 6, 7, 0]	10	1	1
r_2	(9, 1)	[1, 1, 1, 1]	[8, 8, 7, 5]	10	1	1
r_3	(9, 5)	[0, 1, 1, 1]	[0, 6, 7, 9]	10	1	1
r_4	(1, 5)	[1, 1, 0, 1]	[8, 7, 0, 4]	10	1	1

second one, we combined artificial bee colony optimization and quantum genetic algorithms. Such metaheuristics are well known for their abilities of finding good solutions. The use of firefly algorithm is justified by the following assumption. Imagine a function with many global optima, among all metaheuristics only this algorithm would converge into all the global optima (fireflies should be distributed uniformly over the entire search space). More precisely, global optima are tasks $M = \{m_1, m_2, \dots\}$ and fireflies are robots $N = \{n_1, n_2, \dots\}$. Only robots which have selected a task (instead all robots) are considered to compute its possible overlapping partitions, which results in the reduction of the number of robots considered for each task. In other words, the set M is split into $|M|$ singletons and the set N is split into small groups using the firefly algorithm. Then, each singleton and its corresponding group will constitute the input of the second stage. Now, artificial bee colony optimization (known for its good search space exploration) and quantum genetic algorithms (known for their good search space exploitation) will offer a good allocation for each task being allocated.

3 Simulation and result discussion

We evaluate the obtained results after simulating the proposed algorithms on some numerical data. We assume that all tasks are known beforehand—however, algorithms can also handle the case where tasks are progressively discovered (dynamic).

3.1 Generation of simulation data

The numerical data used for our simulations are randomly generated on a 2D grid of 100×100 cells. The number of used resources is 10. The speed, energy level, and aging factor of bidders are supposed to be constant for all.

These data have been divided into three datasets—i.e., dataset 1, dataset 2, and dataset 3. The first dataset has from 10 to 50 bidders and 100 tasks—bidders are incremented by 10. The second dataset has from 10 to 100 tasks and 10 bidders—tasks are incremented by 10. The third dataset has from 10 to 100 tasks and 50 bidders—tasks are incremented by 10. To find out the effect of the number of bidders on allocation performance in relation to the number of tasks,

Table 4 Example of tasks' data

ID task	Location	Resources	Duration
t_1	(3, 3)	[1, 1, 1, 0]	10
t_2	(7, 3)	[1, 1, 0, 1]	12

dataset 1 is used. We used datasets 2 and 3 to measure how the number of tasks affect the allocation performance, with the respect to the number of bidders. Finally, the data format used for bidders and tasks is exemplified in Tables 3 and 4.

3.2 Results and analyzes

In order to show the effectiveness of our solution, we compared its performance with the work presented in Zitouni and Maamri [40]. We evaluated the performance of both solutions in terms of allocation time and number of allocated tasks. The names “FA-QABC-MRTA” and “FA-POWERSET-MRTA” are adopted to refer our solution and the solution proposed in Zitouni and Maamri [40], respectively.

The “FA-POWERSET-MRTA” [40] solution also uses two stages to address the MRTA problem. In the first stage, it behaves exactly like our solution, i.e., bidders use the firefly algorithm for tasks' choice. In the second stage, it calculates all subsets composed of all resources offered by bidders—for the considered task, then it chooses the one representing the optimal allocation. For the example depicted in Fig. 1, the solution proposed by “FA-POWERSET-MRTA” must consider $2^{4+3+5+6}$ different subsets and choose the optimal allocation, i.e., the subset optimizing the criterion taken into account. Imagine its limit when the numbers of bidders and resources are large.

Figures 3 and 4 show a comparison of “FA-QABC-MRTA” and “FA-POWERSET-MRTA” in terms of the allocation time and the utility values, respectively. We used the dataset 1—100 tasks and the number of bidders varies from 10 to 50. First, we notice that increasing the number of bidders does not greatly improve task allocation time for both solutions—i.e., on average, we have 20.199 s for “FA-POWERSET-MRTA” and 5.261 s for “FA-QABC-MRTA,” no matter the number of bidders. Therefore, a large number of tasks are allocated by both methods even when a small number of bidders are used. However, we clearly

Fig. 3 Comparison of the “FA-QABC-MRTA” and “FA-POWERSET-MRTA” solutions for dataset 1—allocation time

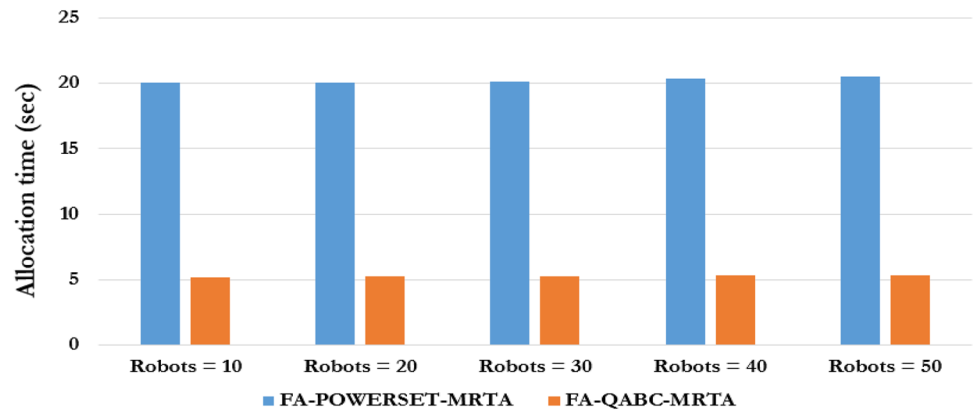


Fig. 4 Comparison of the “FA-QABC-MRTA” and “FA-POWERSET-MRTA” solutions for dataset 1—utility values

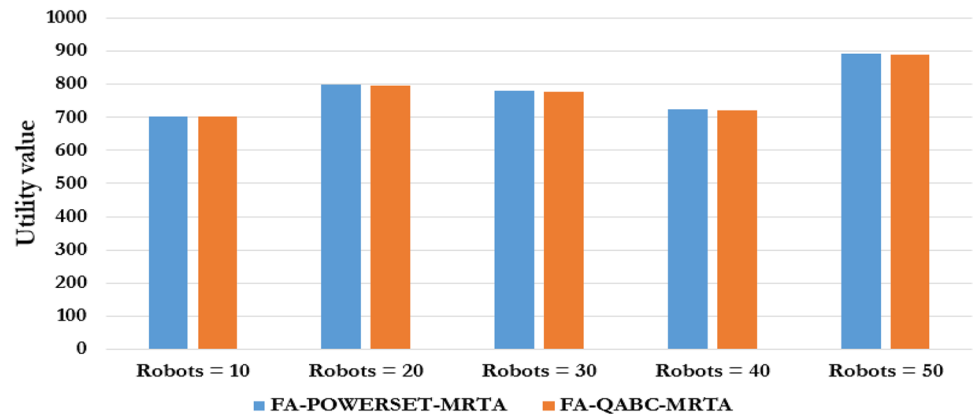
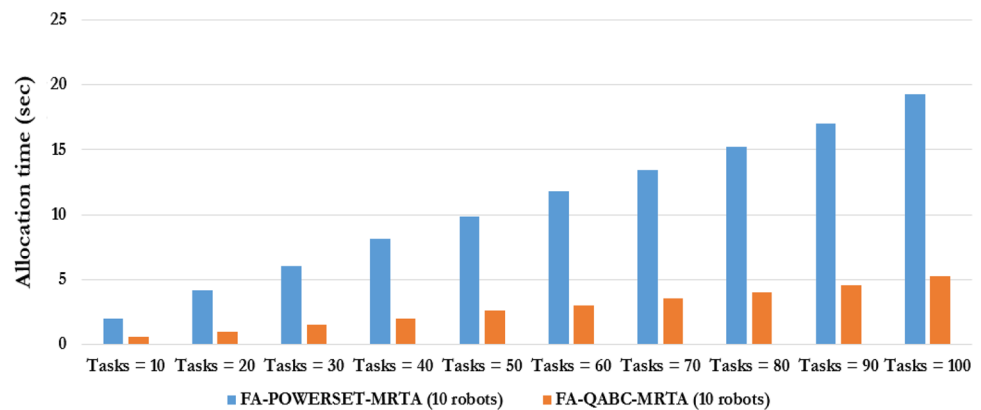


Fig. 5 Comparison of the “FA-QABC-MRTA” and “FA-POWERSET-MRTA” solutions for dataset 2—allocation time



see that “FA-QABC-MRTA” is significantly better than “FA-POWERSET-MRTA” in terms of allocation time—i.e., approximately 15 s of difference for each configuration. On the other hand, we also compared the utility values of these solutions. We notice that they are very close, that is to say, qualities of our allocations are close to qualities of optimal allocations.

Figures 5 and 6 also show a comparison of “FA-QABC-MRTA” and “FA-POWERSET-MRTA” in terms of the allocation time. We used 10 bidders (dataset 2) and 50 bidders (dataset 3), respectively. The number of tasks varies from 10 to 100 in both cases. Again, we notice that increasing the

number of bidders does not greatly improve the allocation time, no matter the number of tasks—with 10 or 50 bidders, we have approximately same times for both solutions. Naturally, the allocation time should increase as the number of tasks increases. However, the solution is efficient with respect to time complexity since the increase of allocation time is almost linear. It should be noted that allocation time that is shown in these figures encompass all actions required for task allocation—i.e., communications between bidders and auctioneer, global allocation, and local allocation.

We graphically show the impact of the number of used bidders on the number of allocated tasks. Figure 7 shows

Fig. 6 Comparison of the “FA-QABC-MRTA” and “FA-POWERSET-MRTA” solutions for dataset 3—allocation time

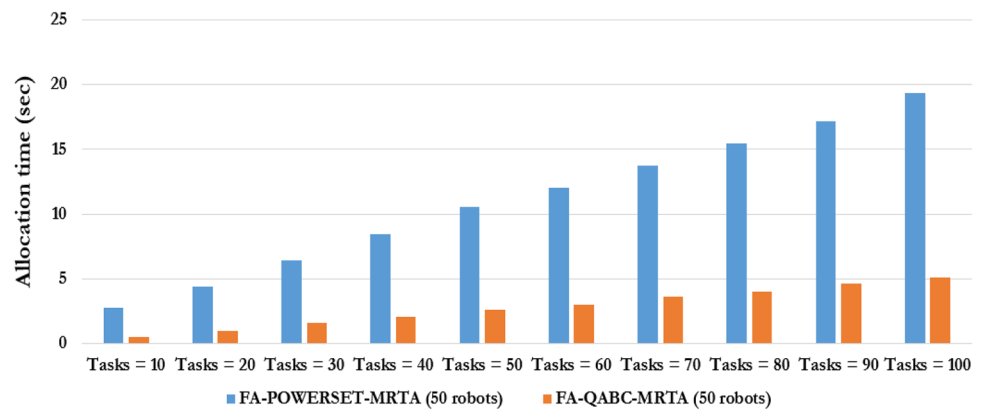
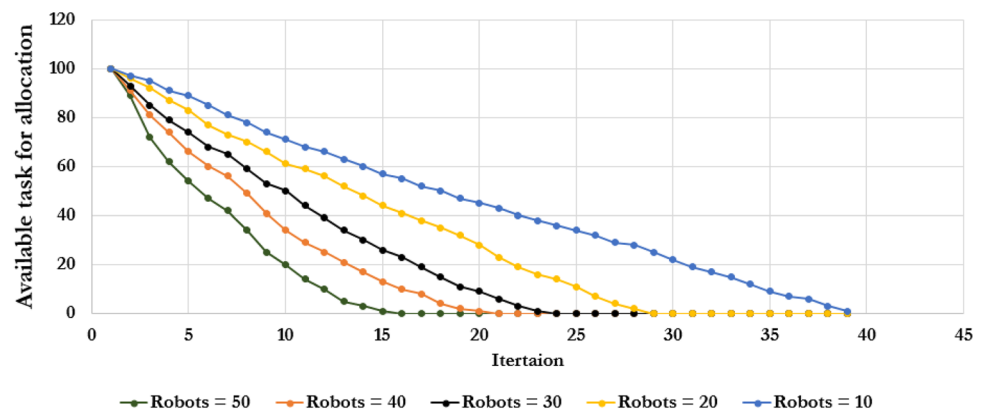


Fig. 7 Number of allocated tasks according to the number of used bidders



the number of allocated tasks per cycle, according to the number of used bidders. We used 100 tasks and the number of bidders varies from 10 to 50. We notice that all tasks are allocated, no matter the used configuration. So, our solution is efficient in terms of the number of allocated tasks. On the other hand, we confirm that increasing the number of bidders only increases the number of tasks allocated per iteration, but does not minimize the allocation time.

4 Conclusion and perspectives

We proposed a solution to address the MRTA problem. The solution uses the auctions with two robot types: auctioneer (one instance) and bidders (several instances). In addition, the solution implements two optimization types: global (performed by bidders) and local (performed by the auctioneer). A comparison was made with the work [40] and showed the superiority of our solution, in terms of allocation time. The result discussion shows that our solution solves the MRTA problem in a linear trend and is efficient in terms of the number of allocated tasks—the rate of allocated tasks is 100%, no matter the adopted configuration. Finally, our solution supports cases where all tasks are known beforehand or discovered dynamically. As perspectives, we plan to further improve our solution by using energetic, temporal, and spa-

tial constraints on tasks and robots. Also, we would use real robots.

References

1. Barnhart C, Johnson EL, Nemhauser GL, Savelsbergh MWP, Vance PH (1996) Branch-and-price: column generation for solving huge integer programs. *Oper Res* 46:316–329
2. Berhault M, Huang H, Keskinocak P, Koenig S, Elmaghraby W, Griffin P, Kleywegt A (2003) Robot exploration with combinatorial auctions. In: *Proceedings 2003 IEEE/RSJ international conference on intelligent robots and systems (IROS 2003)* (Cat. No.03CH37453), vol 2, pp 1957–1962. <https://doi.org/10.1109/IROS.2003.1248932>
3. Botelho SC, Alami R (1999) M+: a scheme for multi-robot cooperation through negotiated task allocation and achievement. In: *Proceedings 1999 IEEE International Conference on Robotics and Automation* (Cat. No. 99CH36288C), vol 2, pp 1234–1239. <https://doi.org/10.1109/ROBOT.1999.772530>
4. Chien S, Barrett A, Estlin T, Rabideau G (2000) A comparison of coordinated planning methods for cooperating rovers. In: *Proceedings of the fourth international conference on autonomous agents, AGENTS '00*. ACM, New York, pp 100–101. <https://doi.org/10.1145/336595.337057>
5. Dias MB (2004) *Traderbots: a new paradigm for robust and efficient multirobot coordination in dynamic environments*. PhD Thesis, Carnegie Mellon University, Pittsburgh, PA
6. Gerkey BP, Mataric MJ (2004) A formal analysis and taxonomy of task allocation in multi-robot systems. *Int J Robot Res* 23(9):939–954. <https://doi.org/10.1177/0278364904045564>

7. Gerkey BP, Mataric MJ (2002) Sold!: auction methods for multi-robot coordination. *IEEE Trans Robot Autom* 18(5):758–768. <https://doi.org/10.1109/TRA.2002.803462>
8. Guerrero J, Oliver G (2003) Multi-robot task allocation strategies using auction-like mechanisms. *Artif Res Dev Front Artif Intell Appl* 100:111–122
9. Han Y, Li D, Chen J, Yang X, Hu Y, Zhang G (2010) A multi-robots task allocation algorithm based on relevance and ability with group collaboration, vol 3. <https://doi.org/10.22266/ijies2010.0630.05>
10. Hussein A, Khamis A (2013) Market-based approach to multi-robot task allocation. In: *International conference on individual and collective behaviors in robotics (ICBR)*, pp 69–74. <https://doi.org/10.1109/ICBR.2013.6729278>
11. Hussein A, Adel M, Bakr M, Shehata OM, Khamis A (2014) Multi-robot task allocation for search and rescue missions. *J Phys Conf Ser* 570(5):052006
12. Jones EG, Dias MB, Stentz A (2011) Time-extended multi-robot coordination for domains with intra-path constraints. *Autonom Robots* 30(1):41–56. <https://doi.org/10.1007/s10514-010-9202-3>
13. Karaboga D, Basturk B (2007) A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. *J Global Optim* 39(3):459–471. <https://doi.org/10.1007/s10898-007-9149-x>
14. Katriel J (2008) On a generalized recurrence for bell numbers. *J Integer Sequ* 11(2):3
15. Khamis A, Hussein A, Elmogy A (2015) Multi-robot task allocation: a review of the state-of-the-art. Springer, Berlin, pp 31–51. <https://doi.org/10.1007/978-3-319-18299-52>
16. Koenig S, Tovey C, Zheng X, Sungur I (2007) Sequential bundle-bid single-sale auction algorithms for decentralized control. In: *Proceedings of the 20th international joint conference on artificial intelligence, IJCAI'07*. Morgan Kaufmann Publishers Inc., San Francisco, pp 1359–1365
17. Koes M, Nourbakhsh I, Sycara K (2006) Constraint optimization coordination architecture for search and rescue robotics. In: *Proceedings 2006 IEEE international conference on robotics and automation, 2006. ICRA 2006*, pp 3977–3982. <https://doi.org/10.1109/ROBOT.2006.1642312>
18. Korsah GA (2011) Exploring bounded optimal coordination for heterogeneous teams with cross-schedule dependencies. PhD Thesis, Carnegie Mellon University, Pittsburgh, PA
19. Korsah GA, Stentz A, Dias MB (2013) A comprehensive taxonomy for multi-robot task allocation. *Int J Robot Res* 32(12):1495–1512. <https://doi.org/10.1177/0278364913496484>
20. Lagoudakis MG, Markakis E, Kempe D, Keskinocak P, Kleywegt A, Koenig S, Tovey C, Meyerson A, Jain S (2005) Auction-based multi-robot routing. In: *Proceedings of robotics: science and systems*, Cambridge, USA. <https://doi.org/10.15607/RSS.2005.I.045>
21. Lahoz-Beltra R (2016) Quantum genetic algorithms for computer scientists. *Computers* 5(4):24
22. Lemaire T, Alami R, Lacroix S (2004) A distributed tasks allocation scheme in multi-uav context. In: *2004 IEEE international conference on robotics and automation, 2004. Proceedings. ICRA '04*, vol 4, pp 3622–3627. <https://doi.org/10.1109/ROBOT.2004.1308816>
23. Liu L, Zhiqiang Z (2005) Combinatorial bids based multi-robot task allocation method. In: *Proceedings of the 2005 IEEE international conference on robotics and automation*, pp 1145–1150. <https://doi.org/10.1109/ROBOT.2005.1570270>
24. Mackenzie DC (2003) Collaborative tasking of tightly constrained multi-robot missions. In: *Multi-robot systems: from swarms to intelligent automata: proceedings of the 2003 international workshop on multi-robot systems*. Kluwer Academic Publishers
25. Melvin J, Keskinocak P, Koenig S, Tovey CA, Ozkaya BY (2007) Multi-robot routing with rewards and disjoint time windows. In: *IROS*, pp 2332–2337
26. Nunes E, Manner M, Mitiche H, Gini M (2017) A taxonomy for task allocation problems with temporal and ordering constraints. *Robot Auton Syst* 90(C):55–70. <https://doi.org/10.1016/j.robot.2016.10.008>
27. Orden A (1956) The transshipment problem. *Manag Sci* 2(3):276–285. <https://doi.org/10.1287/mnsc.2.3.276>
28. Parker LE, Fang T (2006) Building multirobot coalitions through automated task solution synthesis. *Proc IEEE* 94(7):1289–1305. <https://doi.org/10.1109/JPROC.2006.876933>
29. Shehory O, Kraus S (1995) Task allocation via coalition formation among autonomous agents. In: *Proceedings of the 14th international joint conference on artificial intelligence, IJCAI'95*, vol 1. Morgan Kaufmann Publishers Inc., San Francisco, pp 655–661
30. Shehory O, Kraus S (1998) Methods for task allocation via agent coalition formation. *Artif Intell* 101(1):165–200. [https://doi.org/10.1016/S0004-3702\(98\)00045-9](https://doi.org/10.1016/S0004-3702(98)00045-9)
31. Shiroma PM, Campos MFM (2009) Comutar: A framework for multi-robot coordination and task allocation. In: *2009 IEEE/RSJ international conference on intelligent robots and systems*, pp 4817–4824. <https://doi.org/10.1109/IROS.2009.5354166>
32. Simmons RG, Apfelbaum D, Burgard W, Fox D, Moors M, Thrun S, Younes HLS (2000) Coordination for multi-robot exploration and mapping. In: *Proceedings of the seventeenth national conference on artificial intelligence and twelfth conference on innovative applications of artificial intelligence*. AAAI Press, pp 852–858
33. Smith RG (1980) The contract net protocol: high-level communication and control in a distributed problem solver. *IEEE Trans Comput* C-29(12):1104–1113. <https://doi.org/10.1109/TC.1980.1675516>
34. Tang F, Parker LE (2007) A complete methodology for generating multi-robot task solutions using asymptre-d and market-based task allocation. In: *Proceedings 2007 IEEE international conference on robotics and automation*, pp 3351–3358. <https://doi.org/10.1109/ROBOT.2007.363990>
35. Vail D, Veloso M (2003) Multi-robot dynamic role assignment and coordination through shared potential fields. In: A Schultz LP, Schneider F (eds) *Multi-robot systems*
36. Vig L, Adams JA (2006) Multi-robot coalition formation. *IEEE Trans Rob* 22(4):637–649. <https://doi.org/10.1109/TRO.2006.878948>
37. Yang XS (2008) *Nature-inspired metaheuristic algorithms*. Luniver Press, Bristol
38. Yang XS (2009) Firefly algorithms for multimodal optimization. In: Watanabe O, Zeugmann T (eds) *Stochastic algorithms: foundations and applications*. Springer, Berlin, pp 169–178
39. Zitouni F, Maamri R (2018a) Cooperative learning-agents for task allocation problem. In: Auer ME, Tsiatsos T (eds) *Interactive mobile communication technologies and learning*. Springer, Cham, pp 952–968
40. Zitouni F, Maamri R (2018b) FA-SETPOWER-MRTA: a solution for solving the multi-robot task allocation problem. In: *Computational intelligence and its applications—6th IFIP TC 5 international conference, CHIA 2018, Oran, Algeria, May 8–10, 2018, Proceedings*, pp 317–328. <https://doi.org/10.1007/978-3-319-89743-128>
41. Zlot RM (2006) An auction-based approach to complex task allocation for multirobot teams. PhD Thesis, Pittsburgh, PA, USA, aAI3250901

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.